

Vision service API

The vision service enables your machine to use its on-board [cameras](#) to intelligently see and interpret the world around it. While the camera component lets you access what your machine’s camera sees, the vision service allows you to interpret your image data.

The vision service API allows you to get detections, classifications, or point cloud objects, depending on the ML model the vision service is using.

The vision service supports the following kinds of operations:

- [Detections](#)
- [Classifications](#)
- [Segmentations](#)

Detections

2D Object Detection is the process of taking a 2D image from a camera and identifying and drawing a box around the distinct “objects” of interest in the scene. Any camera that can return 2D images can use 2D object detection.

You can use different types of detectors, both based on heuristics and machine learning, for any object you may need to identify.

The returned detections consist of the bounding box around the identified object, as well as its label and confidence score:

- `x_min` , `y_min` , `x_max` , `y_max` (int): specify the bounding box around the object.
- `class_name` (string): specifies the label of the found object.
- `confidence` (float): specifies the confidence of the assigned label. Between `0.0` and `1.0` , inclusive.

Supported API methods:

- [GetDetections\(\)](#)
- [GetDetectionsFromCamera\(\)](#)



Classifications

2D Image Classification is the process of taking a 2D image from a camera and deciding which class label, out of many, best describes the given image. Any camera that can return 2D images can use 2D image classification.

The class labels used for classification vary and depend on the machine learning model and how it was trained.

The returned classifications consist of the image’s class label and confidence score.

- `class_name` (string): specifies the label of the found object.
- `confidence` (float): specifies the confidence of the assigned label. Between `0.0` and `1.0` , inclusive.

Supported API methods:

- [GetClassifications\(\)](#)
- [GetClassificationsFromCamera\(\)](#)

Segmentations

3D Object Segmentation is the process of separating and returning a list of the identified “objects” from a 3D scene. The “objects” are usually a list of point clouds with associated metadata, like the label, the 3D bounding box, and center coordinates of the object.

3D object segmentation is useful for obstacle detection. See our guide [Navigate with a Rover Base](#) for an example of automating obstacle avoidance with 3D object segmentation for obstacle detection.

Any camera that can return 3D pointclouds can use 3D object segmentation.

Supported API methods:

- [GetObjectPointClouds\(\)](#)

The [vision service](#) supports the following methods:

Method Name	Description
GetDetectionsFromCamera	Get a list of detections from the next image from a specified camera using a configured detector.
GetDetections	Get a list of detections from a given image using a configured detector.
GetClassificationsFromCamera	Get a list of classifications from the next image from a specified camera using a configured classifier.
GetClassifications	Get a list of classifications from a given image using a configured classifier.
GetObjectPointClouds	Get a list of 3D point cloud objects and associated metadata in the latest picture from a 3D camera (using a specified segmenter).
CaptureAllFromCamera	Get the next image, detections, classifications, and objects all together, given a camera name.
Reconfigure	Reconfigure this resource.
DoCommand	Execute model-specific commands that are not otherwise defined by the service API.
GetResourceName	Get the <code>ResourceName</code> for this instance of the vision service with the given name.
GetProperties	Fetch information about which vision methods a given vision service supports.
Close	Safely shut down the resource and prevent further use.

Establish a connection

To get started using Viam's SDKs to connect to and control your machine, go to your machine's page on the [Viam app](#), navigate to the **CONNECT** tab's **Code sample** page, select your preferred programming language, and copy the sample code.

To show your machine's API key in the sample code, toggle **Include API key**.

CAUTION: KEEP YOUR API KEY AND MACHINE ADDRESS SAFE

We strongly recommend that you add your API key and machine address as an environment variable. Anyone with these secrets can access your machine, and the computer running your machine.

When executed, this sample code creates a connection to your machine as a client.

The following examples assume that you have a machine configured with a [camera](#) and a vision service [detector](#), [classifier](#) or [segmenter](#).

[Python](#)

[Go](#)

```
from viam.services.vision import VisionClient
```

API

GetDetectionsFromCamera

Get a list of detections from the next image from a specified camera using a configured [detector](#).

Python	Go	Flutter
------------------------	--------------------	-------------------------

Parameters:

- `camera_name` ([str](#)) (required): The name of the camera to use for detection.
- `extra` (Mapping[[str](#), Any]) (optional): Extra options to pass to the underlying RPC call.
- `timeout` ([float](#)) (optional): An option to set how long to wait (in seconds) before calling a time-out and closing the underlying RPC call.

Returns:

- ([List\[viam.proto.service.vision.Detection\]](#)): A list of 2D bounding boxes, their labels, and the confidence score of the labels, around the found objects in the next 2D image from the given camera, with the given detector applied to it.

Raises:

- (ViamError): Raised if given an image without a specified width and height.

Example:

```
1 my_detector = VisionClient.from_robot(robot=machine, "my_detector")
2
3 # Get detections for the next image from the specified camera
4 detections = await my_detector.get_detections_from_camera("my_camera")
```

For more information, see the [Python SDK Docs](#).

GetDetections

Get a list of detections from a given image using a configured [detector](#).

Python	Go	Flutter
------------------------	--------------------	-------------------------

Parameters:

- `image` ([viam.media.video.ViamImage](#)) (required): The image to get detections for.
- `extra` (Mapping[[str](#), Any]) (optional): Extra options to pass to the underlying RPC call.
- `timeout` ([float](#)) (optional): An option to set how long to wait (in seconds) before calling a time-out and closing the underlying RPC call.

Returns:

- ([List\[viam.proto.service.vision.Detection\]](#)): A list of 2D bounding boxes, their labels, and the confidence score of the labels, around the found objects in the next 2D image from the given camera, with the given detector applied to it.

Raises:

- (ViamError): Raised if given an image without a specified width and height.

Example:

```
1 my_camera = Camera.from_robot(robot=machine, "my_camera")
2 my_detector = VisionClient.from_robot(robot=machine, "my_detector")
3
4 # Get an image from the camera
5 img = await my_camera.get_image()
6
7 # Get detections for that image
8 detections = await my_detector.get_detections(img)
```

For more information, see the [Python SDK Docs](#).

GetClassificationsFromCamera

Get a list of classifications from the next image from a specified camera using a configured [classifier](#).

Python	Go	Flutter
------------------------	--------------------	-------------------------

Parameters:

- `camera_name` ([str](#)) (required): The name of the camera to use for detection.
- `count` ([int](#)) (required): The number of classifications desired.
- `extra` (Mapping[[str](#), Any]) (optional): Extra options to pass to the underlying RPC call.
- `timeout` ([float](#)) (optional): An option to set how long to wait (in seconds) before calling a time-out and closing the underlying RPC call.

Returns:

- ([List\[viam.proto.service.vision.Classification\]](#)): The list of Classifications.

Example:

```
1 my_classifier = VisionClient.from_robot(robot=machine, "my_classifier")
2
3 # Get the 2 classifications with the highest confidence scores for the next image from the camera
4 classifications = await my_classifier.get_classifications_from_camera(
5     "my_camera", 2)
```

For more information, see the [Python SDK Docs](#).

GetClassifications

Get a list of classifications from a given image using a configured [classifier](#).

Python	Go	Flutter
------------------------	--------------------	-------------------------

Parameters:

- `image` ([viam.media.video.ViamImage](#)) (required): The image to get detections for.
- `count` ([int](#)) (required): The number of classifications desired.
- `extra` (Mapping[[str](#), Any]) (optional): Extra options to pass to the underlying RPC call.
- `timeout` ([float](#)) (optional): An option to set how long to wait (in seconds) before calling a time-out and closing the underlying RPC call.

Returns:

- ([List\[viam.proto.service.vision.Classification\]](#)): The list of Classifications.

Example:

```
1 my_camera = Camera.from_robot(robot=machine, "my_camera")
2 my_classifier = VisionClient.from_robot(robot=machine, "my_classifier")
3
4 # Get an image from the camera
5 img = await my_camera.get_image()
6
7 # Get the 2 classifications with the highest confidence scores for the image
8 classifications = await my_classifier.get_classifications(img, 2)
```

For more information, see the [Python SDK Docs](#).

GetObjectPointClouds

Get a list of 3D point cloud objects and associated metadata in the latest picture from a 3D camera (using a specified [segmenter](#)).

Python	Go	Flutter
------------------------	--------------------	-------------------------

Parameters:

- `camera_name` ([str](#)) (required): The name of the camera.
- `extra` (Mapping[[str](#), Any]) (optional): Extra options to pass to the underlying RPC call.
- `timeout` ([float](#)) (optional): An option to set how long to wait (in seconds) before calling a time-out and closing the underlying RPC call.

Returns:

- ([List\[viam.proto.common.PointCloudObject\]](#)): The pointcloud objects with metadata.

Example:

```
1 import numpy as np
2 import open3d as o3d
3
4 my_segmenter = VisionClient.from_robot(robot=machine, "my_segmenter")
5 # Get the objects from the camera output
6 objects = await my_segmenter.get_object_point_clouds("my_camera")
7 # write the first object point cloud into a temporary file
8 with open("/tmp/pointcloud_data.pcd", "wb") as f:
9     f.write(objects[0].point_cloud)
10 pcd = o3d.io.read_point_cloud("/tmp/pointcloud_data.pcd")
11 points = np.asarray(pcd.points)
```

For more information, see the [Python SDK Docs](#).

CaptureAllFromCamera

Get the next image, detections, classifications, and objects all together, given a camera name. Used for visualization.

Python	Go
------------------------	--------------------

Parameters:

- `camera_name` ([str](#)) (required): The name of the camera to use for detection.
- `return_image` ([bool](#)) (required): Ask the vision service to return the camera’s latest image.
- `return_classifications` ([bool](#)) (required): Ask the vision service to return its latest classifications.
- `return_detections` ([bool](#)) (required): Ask the vision service to return its latest detections.
- `return_object_point_clouds` ([bool](#)) (required): Ask the vision service to return its latest 3D segmentations.
- `extra` (Mapping[[str](#), Any]) (optional): Extra options to pass to the underlying RPC call.
- `timeout` ([float](#)) (optional): An option to set how long to wait (in seconds) before calling a time-out and closing the underlying RPC call.

Returns:

- ([viam.services.vision.vision.CaptureAllResult](#)): A class that stores all potential returns from the vision service. It can return the image from the camera along with its associated detections, classifications, and objects, as well as any extra info the model may provide.

Example:

```
1 my_detector = VisionClient.from_robot(machine, "my_detector")
2
3 # Get the captured data for a camera
4 result = await my_detector.capture_all_from_camera(
5     "my_camera",
6     return_image=True,
7     return_detections=True,
8 )
9 image = result.image
10 detections = result.detections
```

For more information, see the [Python SDK Docs](#).

Reconfigure

Reconfigure this resource. Reconfigure must reconfigure the resource atomically and in place.

[Go](#)

Parameters:

- `ctx` ([Context](#)): A Context carries a deadline, a cancellation signal, and other values across API boundaries.
- `deps` ([Dependencies](#)): The resource dependencies.
- `conf` ([Config](#)): The resource configuration.

Returns:

- [\(error\)](#): An error, if one occurred.

For more information, see the [Go SDK Docs](#).

DoCommand

Execute model-specific commands that are not otherwise defined by the service API. For built-in service models, any model-specific commands available are covered with each model's documentation. If you are implementing your own vision service and add features that have no built-in API method, you can access them with `DoCommand`.

[Python](#)

[Go](#)

[Flutter](#)

Parameters:

- `command` (Mapping[[str](#), ValueTypes]) (required): The command to execute.
- `timeout` ([float](#)) (optional): An option to set how long to wait (in seconds) before calling a time-out and closing the underlying RPC call.

Returns:

- (Mapping[[str](#), viam.utils.ValueTypes]): Result of the executed command.

Example:

```
1 my_vision_svc = VisionClient.from_robot(robot=machine, "my_vision_svc")
2
3 my_command = {
4     "cmd": "dosomething",
5     "someparameter": 52
6 }
7
8 await my_vision_svc.do_command(command=my_command)
```

For more information, see the [Python SDK Docs](#).

GetResourceName

Get the `ResourceName` for this instance of the vision service with the given name.

[Python](#)

[Flutter](#)

Parameters:

- `name` ([str](#)) (required): The name of the Resource.

Returns:

- ([viam.proto.common.ResourceName](#)): The ResourceName of this Resource.

Example:

```
1 | my_vision_svc_name = VisionClient.get_resource_name("my_vision_svc")
```

For more information, see the [Python SDK Docs](#).

GetProperties

Fetch information about which vision methods a given vision service supports.

[Python](#)

[Go](#)

[Flutter](#)

Parameters:

- `extra` (Mapping[[str](#), Any]) (optional): Extra options to pass to the underlying RPC call.
- `timeout` ([float](#)) (optional): An option to set how long to wait (in seconds) before calling a time-out and closing the underlying RPC call.

Returns:

- ([Vision.Properties](#)): The properties of the vision service.

Example:

```
1 | my_detector = VisionClient.from_robot(robot=machine, "my_detector")
2 | properties = await my_detector.get_properties()
3 | detections_supported = properties.detections_supported
4 | classifications_supported = properties.classifications_supported
```

For more information, see the [Python SDK Docs](#).

Close

Safely shut down the resource and prevent further use.

[Python](#)

[Go](#)

Parameters:

- None.

Returns:

- None.

Example:

```
1 my_vision_svc = VisionClient.from_robot(robot=machine, name="my_vision_svc")
2 await my_vision_svc.close()
```

For more information, see the [Python SDK Docs](#).

⏪

PREVIOUS

[Data Management](#)

[ML Model](#)

NEXT

⏩